

2. Requirement Analysis Phase

ComicCraft – AI Comic Story Creator Using Gemini Models

Defining the functional, interface, and technical requirements

1. Purpose

This phase translates the ComicCraft concept into requirements that guide implementation and testing. The requirements below are based on the supplied project guide.

2. Functional Requirements

- Collect a story prompt, character name, setting, story tone, and art style.
- Generate a structured five-panel outline.
- Expand the outline into narration and character dialogue.
- Generate a comic-style illustration for each panel.
- Match each illustration with its corresponding panel text.
- Display the comic panel-by-panel in a preview page.
- Compile the comic into a downloadable PDF.
- Provide a success page after export.
- Support form-based generation and JSON-based generation.
- Provide a separate image-testing route for developers.

3. User Interface Requirements

The homepage must provide clearly labeled inputs and a direct path to comic generation. The preview page must show each panel's title, image, scene description, caption, and narration. The export success page must confirm completion and provide a way to create another comic.

4. Technical Requirements

The guide specifies Python, FastAPI, Uvicorn, HTML, CSS, Jinja2, Google Gemini APIs, Hugging Face Diffusers, Stable Diffusion, PyTorch, FPDF, Git, and a Python environment manager such as venv. Required Python packages include fastapi, uvicorn, jinja2, python-multipart, google-generativeai, diffusers, transformers, fpdf, Pillow, and accelerate.

5. Interface and API Inputs

The browser form captures the story prompt, character name, setting, tone, and art style. The JSON endpoint uses a Pydantic PromptRequest schema to validate the request body.

6. Outputs and Data

The outline is represented as a list of panel dictionaries. The story is formatted text. Generated images are saved under static/panels. The layout associates panel numbers, image paths, and text. Exported PDFs are saved under static/exports with a timestamped filename.

7. Operational Requirements

The application should run locally through Uvicorn and be accessible in a browser at 127.0.0.1:8000. FastAPI's interactive documentation is available at /docs for API testing. API keys and tokens should be stored in environment variables rather than embedded in source code.

8. Scope Notes

The guide describes single-comic generation and notes that user account management is not included in the current version. User profiles, comic libraries, and multi-page story arcs are described as possible future enhancements.

9. Requirement Analysis Outcome

The phase produces a clear checklist for implementation: input handling, AI generation, image creation, layout assembly, preview rendering, PDF export, API access, and local verification.